École d'ingénieur Denis Diderot
Université Paris Cité

THALES

Implementation of the Language Server Protocol for C/ACSL with Frama-C

August 30, 2024

Internship report

Djamila Mohamed
2023–2024

Supervisor:
Adel Djoudi, Formal Methods Expert

Contents

1 Introduction

1.1 Thales DIS

Thales Digital Identity and Security is the global business unit of Thales which operates in the field of secure software, providing biometric and encryption solutions. Within DIS is the Identity and Biometric Solutions (IBS) business line which is in charge of digital identity (e.g.: cards and passports).

The IBS business line is divided in three subunits: ID Documents Solutions, Digital ID & Verification Solutions and Public Security. Activities described in this report have been conducted within the ID Documents and solutions subunit.

1.2 Context

1.2.1 Frama-C

In Thales DIS, developers use program analyzers such as Frama-C[1].

Frama-C is a toolkit for analyzing source code written in C developed by CEA and Inria. It combines various static and dynamic analysis methods into a single system.

A key feature of Frama-C is its support for the ANSI/ISO C Specification Language (ACSL), a formal language designed to specify the behavior of C programs. ACSL facilitates the precise expression of functional properties through function contracts. It enables both human-readable and machine-processable specifications, ensuring a detailed formal specification of function behavior.

ACSL specification is mainly expressed as function contracts that state preconditions and postconditions of functions. A function precondition must be satisfied when the function is called and a postcondition must be satisfied when the function terminates its execution.

Loop invariants are also needed to specify the expected behavior of loops without unrolling all loop iterations. Figure 1 depicts a C function called set_to_0 that sets all elements of an array to 0 with its associated ACSL specification.

This is a simple example given for the sake of clarity. Advanced ACSL specification may use other constructs such as: predicates, logic functions, ghost code, ect. (c.f. reference to ACSL specification).

Precondition (line 14: requires): The pointer a points to valid memory locations.

Postcondition (line 17: ensures): After the function executes, all elements in the array a from index 0 to n-1 will be 0.

Postcondition (line 15: assigns): Memory locations modified by the function (function side effect on calling sites)

Loop Invariant (line 26): `0 <= i <= n`: The loop variable i always remains within the bounds from 0 to n.

Loop invariant (line 27): For every index j less than i, at each loop iteration and at the end of the loop, the elements $a[j]$ are set to 0.

Loop Variant (line 31): The expression n-i decreases with each iteration, ensuring loop termination.

Loop side effects (line 30: loop assigns ...): The loop modifies the content of array a and variable i that were declared outside of the loop scope.

1.2.2 Problem

Frama-C currently lacks a dedicated integrated development environment (IDE) for editing and browsing C code and ACSL annotations. Writing ACSL specifications often requires manually searching through the C code and ACSL annotations to find relevant declarations and definitions of various terms.

In practice, Frama-C users usually rely on a grep-like utility to search for predicate definitions within large code bases. For example, the command `grep -r 'predicate_name' /path/to/frama-c/libraries` may be used to recursively search for occurrences of `predicate_name` among the Frama-C libraries. While this method can yield results, it requires manual navigation and is prone to inefficiencies, especially in large code bases with extensive libraries. Figure 2 shows an example with the predicate `valid_read_string`.

Searching through potentially large directories and files using grep is time-consuming, especially when dealing with complex libraries with numerous functions and predicates. This method relies heavily on manual effort and familiarity with command-line tools, which can be error-prone and inconvenient.

The lack of efficiencies of the current method emphasizes the necessity for a simpler solution.

Microsoft has designed the Language Server Protocol (LSP) as a common standard to enrich integrated development environments (IDEs) with several language programming features.

An LSP tailored for ACSL code would offer significant improvements by providing features such as code navigation, automatic function and predicate search, contextual information, and interaction between editing,

[1] https://frama-c.com/html/overview.html

```
13  /*@
14    requires \valid(a+(0..n-1));
15    assigns a[0..n-1];
16
17    ensures L : l2: \forall integer i;
18      0 <= i < n ==> a[i] == 0;
19  */
20
21  void set_to_0(int *a, size_t n)
22  {
23    size_t i;
24
25    /*@
26      loop invariant 0 <= i <= n;
27      loop invariant
28      \forall integer j;
29        0 <= j < i ==> a[j] == 0;
30      loop assigns i, a[0..n-1]; // acsl comment example
31      loop variant n-i;
32
33    */
34    for (i = 0; i < n; ++i)
35      a[i] = 0;
36  }
```

Figure 1: Example of ACSL specification for a C function

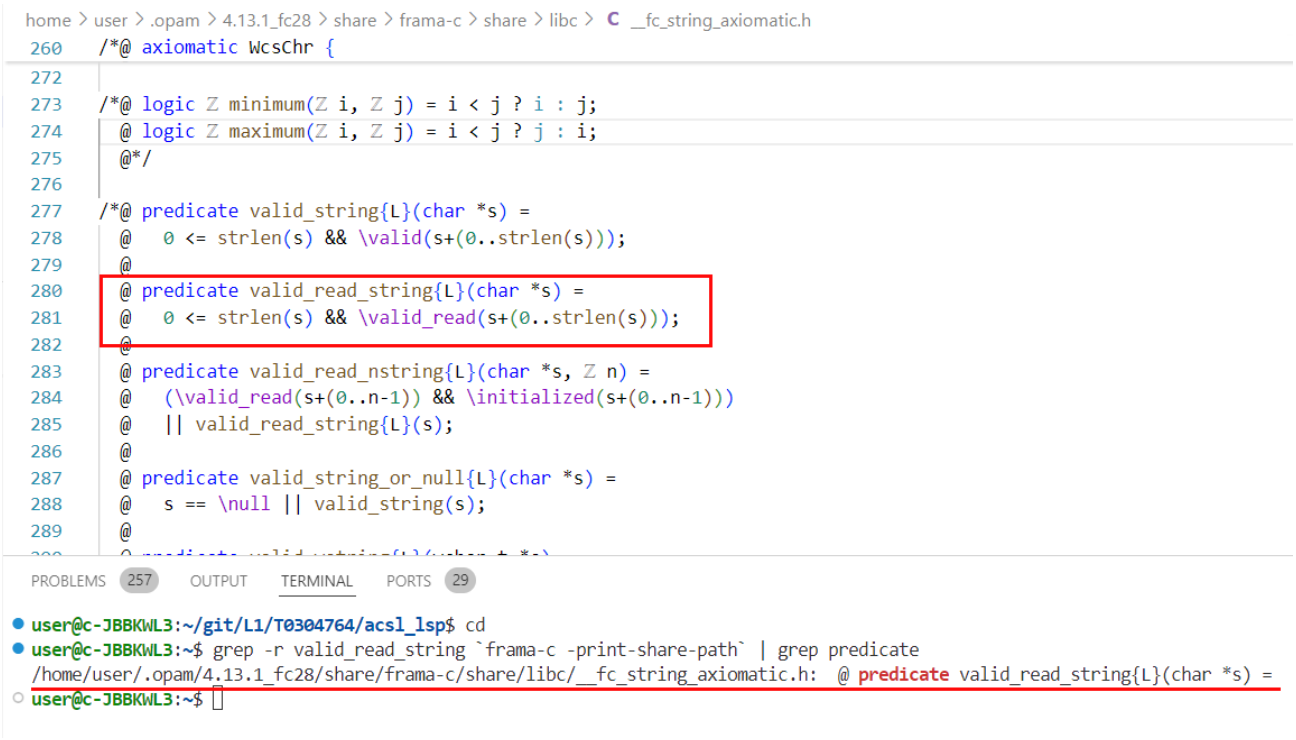


Figure 2: Example of searching predicate definition with grep

parsing and analysis tools. Such a tool would greatly enhance productivity by automating these tasks and reducing the reliance on cumbersome manual search and code parsing processes.

Additionally, those unaccustomed to Frama-C and ACSL annotations can benefit from this solution as user friendly environment to edit C and ACSL code and have a better understanding of the specification language.

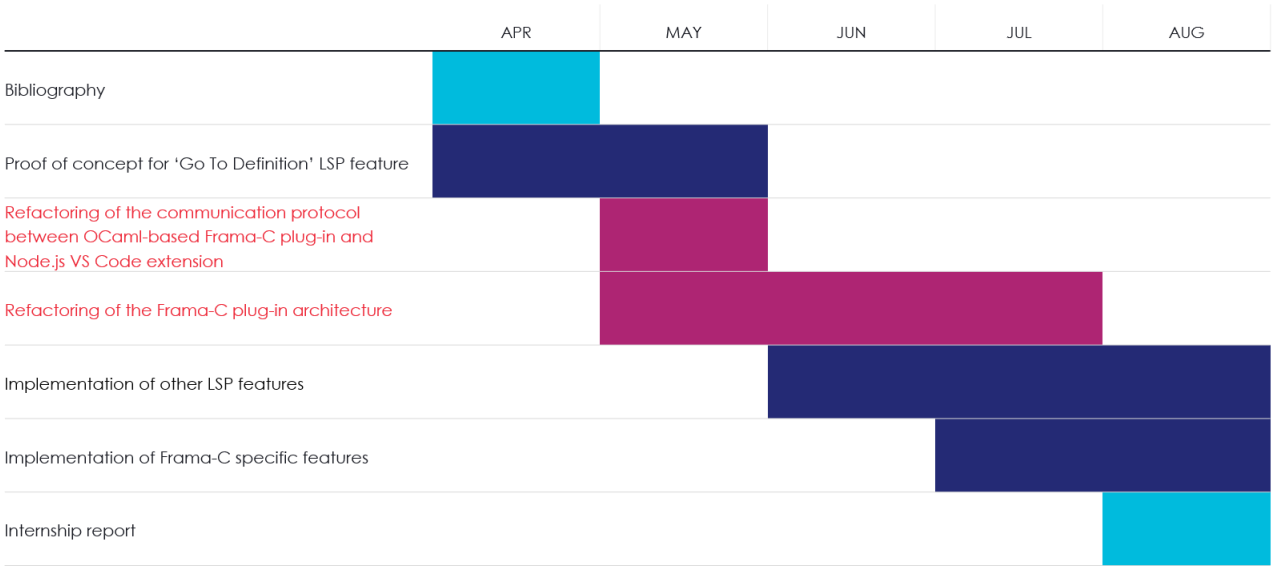
The goal of this project is to develop a software solution to offer language programming features for C/ACSL in IDEs including code navigation, completion suggestions, and interaction with Frama-C code analysis.

1.3 Work organization

The internship started in the end of march (20/03/2024) for a duration of five months. It is divided in seven main tasks, as depicted in figure 3.

- Bibliography: A period of research work was necessary to understand the challenges of the internship. A study of a work similar than the one described in this report was made to understand the subject [1].
- Proof of concept for 'Go To Definition' feature: A first minimal implementation was made to anticipate design choices for the architecture of the final implementation
- Refactoring of the communication protocol between OCaml-based Frama-C plugin and Node.js VS Code extension
- Refactoring of the Frama-C plugin architecture
- Implementation of other LSP features
- Implementation of Frama-C specific features
- Internship report

Figure 3: Work organization of the internship



2 Background

2.1 The Language Server Protocol

2.1.1 Context

Developers working on large scale projects often need a code editing environment with the adequate tooling for the languages they use in their workspace. Multiple solutions such as Eclipse or IntelliJ IDEA exist for Java projects, as well as other popular programming languages. But the complexity of developing one tool for one language and one specific editor is tedious as it forces developers to implement the same language programming features for every code editor with each programming language. To overcome these, a solution was developed: the LSP[2].

The LSP operates through a structured communication system between the client (such as an IDE or code editor) and the server (which provides language-specific features). This communication is facilitated by various types of messages and methods that define the interactions between the client and server.

It is built on top of the JSON-RPC protocol, a remote procedure call protocol that uses JSON as data format. It permits a communication between a client and a server.

Figure 2 illustrates an example of usage of the LSP, it demonstrates the data exchanges between the development tool and the language server.

[2]https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/

2.1.2 LSP Requests

Request Message: A request message is sent by the client to ask the server to perform a specific action. Each request is associated with a particular method, which determines the type of operation being requested. For example, methods like 'textDocument/completion' request code completion suggestions, while 'textDocument/definition' asks for the location of a symbol's definition. These requests expect a response from the server, which will either provide the requested data or indicate an error.

Notification Message: Notification messages are used by the client to inform the server of events or changes in the editor, such as opening, modifying, or closing a document. Unlike request messages, notifications do not expect a response. They keep the server updated on the current state of the workspace. Methods associated with notifications include 'textDocument/didOpen', 'textDocument/didChange', and 'textDocument/didClose'.

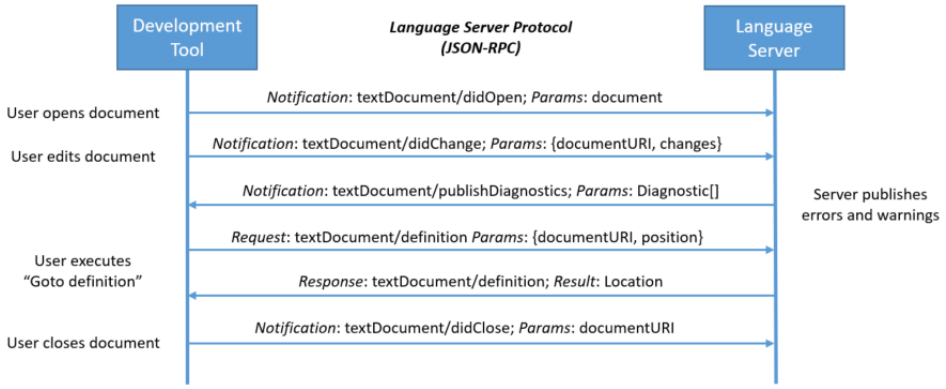


Figure 4: LSP sequence diagram: example of an interaction flow between client and server

2.1.3 LSP Capabilities

The LSP defines sets of features named 'capabilities'. LSP capabilities are boolean values transported in a JSON request that each party declares when the extension is launched. These declarations are needed so that both agree on which features each can provide. Only the features supported by both parties can be used in the editor.

Capabalities can be registered in two ways: statically and dynamically. Static registration is done through the initialize request, so it is done once and for all. Dynamic registration allows language servers to add or remove support for various capabilities at runtime without a reinitialization of the server.

Figure 13 shows a sequence diagram of the first requests exchanged between the client and the server, the user can start using the extension once the server has received the 'initialized' notification by the the client (not to be confused with the 'initialize' request)

2.1.4 Features

LSP features are organized into categories based on the type of operations they perform. These categories include:

Text Document Features: These features deal with operations on text documents, such as opening, editing, saving, and closing files. Examples include `textDocument/didOpen`, `textDocument/didChange`, and `textDocument/didSave`.

Language Features: These features provide language-specific features such as autocompletion, go-to-definition, hover information, and find-references. Examples include `textDocument/completion`, `textDocument/definition`, and `textDocument/hover`.

Workspace Features: These features relate to operations that affect the entire workspace, such as searching for symbols or managing project settings. Examples include workspace/symbol and workspace/configuration.

Table 7 is a listing of all LSP features and the use cases associated with it.

Client option	**Advanced**	**Definition**	**Declaration**	**Display CIL**	**Callgraph**	**Local Metrics**	**Global Metrics**	**Proof Obligations**	**Prove Annotation**	**Diagnostics**
acslLsp		X	X	X	X	X	X	X	X	X

kernel.sourceFiles		X	X				X			X
kernel.includePaths		X	X	X	X	X	X	X	X	X
kernel.macros		X	X	X	X	X	X	X	X	X
kernel.machdep				X	X	X	X	X	X	X
kernel.keepUnusedSpecifiedFunctions		X	X	X	X	X	X	X	X	
kernel.aggressiveMerging		X	X	X	X	X	X	X	X	
kernel.generatedSpecCustom	Yes			X	X	X	X	X		
metrics.output					X	X	X			
callgraph.output					X					
callgraph.roots					X					
callgraph.services					X					
wp.rte									X	
wp.checkMemoryModel	Yes								X	
wp.volatile	Yes								X	
wp.pruning	Yes								X	
wp.prover									X	
wp.timeout									X	
wp.session									X	
diagnostics.wp										X

Table 1: Option details

2.2 Frama-C plugin development

This document provides a brief overview of the key aspects of developing a Frama-C plugin, including OCaml's module system and the process of registration, building, and installing a plugin using Dune.

Overview of OCaml

OCaml is a functional programming language with a rich type system. It organizes code into modules, which can be used to encapsulate and manage related functionalities. Key features of OCaml's module system include:

- Modules: Define and group related types, functions, and values. Modules can be implemented in `.ml` files and their interfaces in `.mli` files.
- Functors: Parameterized modules that can create new modules by applying parameters.
- Interfaces: Define the public signature of a module, specifying which functions and types are exposed.

OCaml's type system helps ensure the correctness of complex static analysis tasks by catching errors at compile time. The language's functional programming features, comprising pattern matching, simplify the manipulation of abstract syntax trees and other data structures crucial for program analysis. Additionally, OCaml's modular system facilitates the organization and extension of Frama-C's various analysis components.

Environment Setup & Documentation

- Frama-C Installation: Ensure Frama-C is installed. The minimum required is version 28.0.
- OCaml Toolchain: Install the OCaml toolchain, including tools like `opam`, `ocamlbuild`, `ocamllsp` and `ocamlfind`.
- It is necessary to have the Frama-C user manual [2] Frama-C plugin development guide [3] beside.

Understanding Frama-C's Architecture

Figure 5 shows how the Frama-C modules are organized.

- Frama-C Kernel: Provides essential services such as Abstract Syntax Tree (AST) management, project handling, and results storage, the kernel services are the main modules we will work with.
- Existing plugins: Depending on the analysis goals, the plugin may need to interact with specific Frama-C modules like CFG or WP. For instance, a plugin designed to analyze the control flow of a program will use the CFG module to traverse and manipulate the CFG.
- API Documentation: The Frama-C API provides an online API where we can browse each plugin's library and better understand its functions

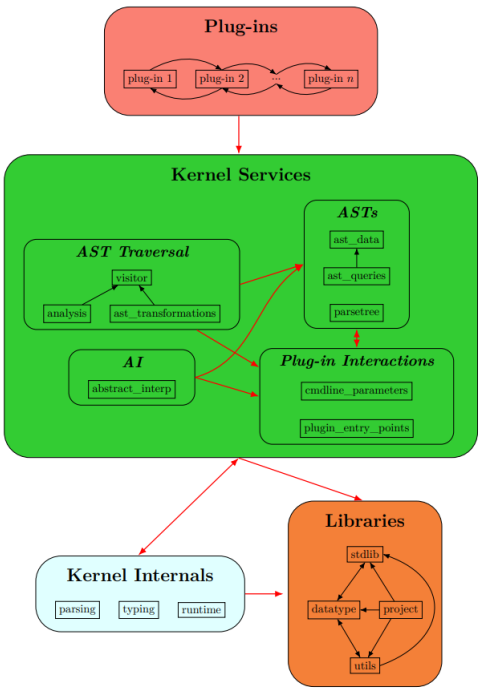


Figure 5: Frama-C Architecture Design

Plug-in Registration

To register a Frama-C plugin, a file that defines the API of the plugin `plugin-name.ml` is created. The typical registration involves defining a module with plugin details and using `Plugin.Register` to integrate the plugin with the Frama-C system. In our case it would be:

```
(struct
       let name = "ACSL LSP"
       let shortname = "lsp"
       let help = "Activates LSP support for ACSL/C"
end)
```

We set the plugin's entry point with the line:

```
let () = Db.Main.extend run
```

Here, Db.Main.extend is used to integrate the plugin's function 'run' into Frama-C's main execution process. The entry point is the starting point of the program's execution, where 'run' manages the plugin's options and starts its operations within the Frama-C environment.

Building with Dune

Dune[3] is a build system for OCaml projects that simplifies the build process. To build a Frama-C plugin using Dune:

– To initiate a project we create a `dune-project` file to define the project settings:

```
(lang dune 3.7)
(using dune_site 0.1)

(name frama-c-acsl_lsp)
(package (name frama-c-acsl_lsp))
```

– Then write a `dune` file to specify build rules, including dependencies and build options:

```
(rule
       (alias frama-c-configure)
       (deps (universe))
       (action (progn
```

[3]https://dune.build/

```
                        (echo "acsl_lsp:" %{lib-available:frama-c-acsl_lsp.core} "\n")
                        (echo " - Frama-C:" %{lib-available:frama-c.kernel} "\n")
                )
        )
)
(include_subdirs unqualified)
(library
        (name acsl_lsp)
        (public_name frama-c-acsl_lsp.core)
        (flags -open Frama_c_kernel :standard)
        (libraries
                frama-c.kernel
                frama-c-wp.core
                frama-c-metrics.core
                frama-c-callgraph.core
        )
)
(plugin
        (name acsl_lsp)
        (libraries frama-c-acsl_lsp.core)
        (site (frama-c plugins))
)
```

– Build the plugin with `dune build`, which compiles the plugin and manages dependencies.

Installing the Plug-in

After building, the plugin is installed using the command: `dune install` This command places the plugin in a directory where Frama-C can find and use it. We verify that the plugin is installed by running `frama-c -lsp` and expect no error messages.

Useful plugins

The following plugins are the ones we will be focusing on in our development process:

Control Flow Graph (CFG): The CFG plugin is central to performing control flow analysis in Frama-C. It allows the construction and manipulation of control flow graphs, which represent all possible paths of execution in a program. Plugins that need to analyze or transform the control flow of a program rely heavily on this module.

Deductive Verification (WP): The WP plugin is used to generate verification conditions using the weakest precondition calculus. These conditions are then checked to ensure that the program adheres to its formal specification, typically expressed in ACSL (ANSI/ISO C Specification Language). This module is crucial for plugins aimed at formal verification or proof-based analysis.

Metrics: This module computes various code metrics, such as cyclomatic complexity and lines of code. These metrics are often used to assess the quality and maintainability of the code, making this module useful for plugins focused on code quality analysis.

Frama-C invariants

Frama-C plugin developers must respect certain invariants, particularly when interacting with the AST. The AST must remain consistent and uncorrupted, as it is central to all analyses. Modifications to the AST should be minimal and carefully managed to avoid disrupting other plugins. The following invariant —as will be explained later— is important in our case: *"An AST must never be modified inside a project."*.

2.3 VS Code Extensions

2.3.1 VS Code extension organization

The VS Code extension is organized with various configuration files and directories. Key components include language-configuration.json for defining a part of the declarative language features, tsconfig.json for TypeScript compiler settings, and package.json for extension configuration. The src directory contains the TypeScript source code that launches the language client, while directories like 'node_modules' and 'out' manage dependencies and compiled outputs.

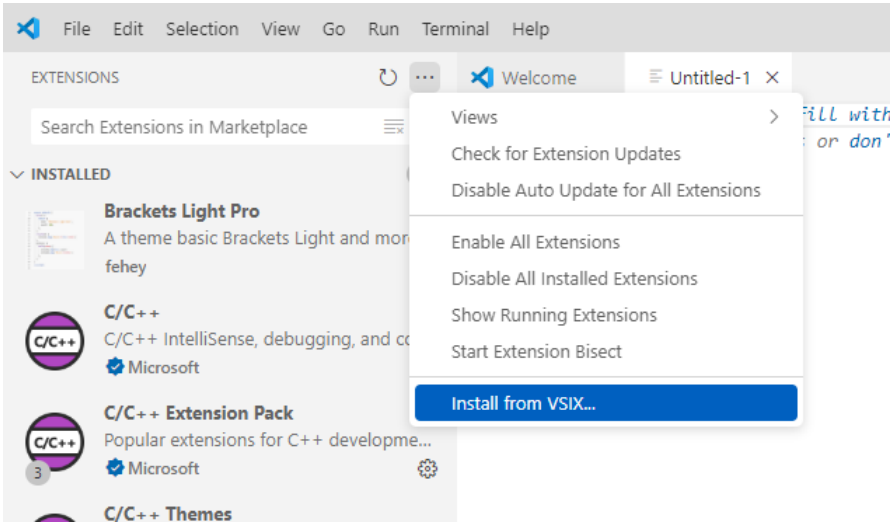
The architecture of VS Code extensions is based on a client-server model, which aligns perfectly with the LSP. VS Code provides an API with TypeScript libraries for developing different types of extensions, particularly,

language extensions. A language extension offers declarative (e.g. syntax highlighting, bracket autoclosing) and programmatic (e.g. hover information, jump to definition) language features.

Installation

The extension is installed by importing the VSIX file located in the client code of the project, as shown in figure 6. The full path to the VSIX file must be provided.

Figure 6: Install VS Code extension



2.3.2 Frama-C extensions

Very few VS Code extensions for Frama-C exist in the marketplace[4]. The only extensions related to Frama-C are for ACSL syntax highlighting made by an independent developer[5].

The first one, 'ACSL-lite'[6] removes syntax highlighting for C code and adds coloring for ACSL annotations, but it has no other feature. The drawback of this extension is that it removes entirely the coloring for C when we also need it.

The second one 'ACSL: ANSI/ISO C Specification Language'[7] has coloring for both ACSL and C. But there are a few fixes to make.

2.3.3 Overlay multiple extensions

The goal is to retain C language features while adding ACSL language support on top. VS Code allows multiple extensions to coexist within the same instance, such as the GitHub Actions with C/C++ extensions. However, it does not permit the activation of different language extensions for the same file type simultaneously. Since the ACSL extension targets the same file extensions as the C language extension in VS Code, it is necessary to reimplement C language features alongside ACSL features within a single extension. While this combined implementation may not offer the full range of functionalities provided by the original C extension, it will ensure the essential features required for effective code navigation.

2.3.4 Language client implementation

Entry point

To develop the VS Code extension language client, the process began with a sample project provided by VS Code, specifically the 'plaintext' extension, a simple code providing autocompletion for predefined words for '.txt' files[8]. This sample was used as a foundational template and was then customized to meet specific requirements. The sample server part was removed as we already have our own language server implemented in OCaml.

The first step involved modifying the sample code to align the extension's functionality with the target language. This included adjusting project configurations to ensure proper interaction between the language server and the client, enabling the correct recognition and processing of the language's syntax and structure.

[4]The Visual Studio marketplace where VS Code extensions can be installed
[5]https://github.com/MisterZurg
[6]https://github.com/MisterZurg/acsl-lite
[7]https://github.com/MisterZurg/acsl-ansi-iso-c-specification-language
[8]https://github.com/microsoft/vscode-extension-samples/tree/main/lsp-sample

For syntax highlighting, TextMate (TM) language grammars were integrated, allowing VS Code to apply syntax highlighting. This required defining patterns and scopes within a .tmLanguage file to highlight various syntax elements such as keywords, strings, and comments. The existing syntax highlighting for ACSL (ACSL-lite) code—as mentioned in 2.3.2— was extracted, fixed, enhanced and combined to C syntax highlighting of official VS Code extension for C and C++[9]

In addition, a language configuration file was provided by the sample code to define language-specific behaviors, including comment styles, bracket matching, and auto-closing pairs.

Table 8 lists all programmatic and declarative language features supported by VS Code, and their implementation status on the server side.

VS Code commands

VS Code commands are custom VS Code-specific features that can be registered for any extension. Commands are declared in the configuration file in the 'commands' section (package.json)(fig. 7). Then they must be set up for a specified menu of the editor (fig. 8) in section 'menus' of package.json and must be implemented and registered in the main entry point (extension.ts) in function 'activate()' (fig. 9).

Figure 7: Example of command declaration in package.json

```
262     "commands": [
263         {
264             "command": "displayCIL",
265             "title": "Display CIL"
266         },
```

Figure 8: Example of command set up in package.json

```
24      "menus": {
25          "editor/context": [
26              {
27                  "command": "displayCIL",
28                  "when": "resourceLangId == acsl"
29              },
```

Figure 9: Example of command implementation in extension.ts

```
64  const displayCIL = commands.registerCommand('displayCIL', async () => {
65      try {
66          const res = await client.sendRequest('displayCIL', window.activeTextEditor.document.fileName);
67          const cilContent = JSON.parse(JSON.stringify(res, null, 1));
68
69
70          // create a new untitled document in a new tab
71          const newUri = Uri.parse('untitled:CIL Representation');
72          const document = await workspace.openTextDocument(newUri);
73          const editor = await window.showTextDocument(document, ViewColumn.Beside, true);
74          await languages.setTextDocumentLanguage(document, 'acsl');
75
76          // delete previous content if any and set the content of the new document
77          editor.edit(editBuilder => {
78              const start = new Position(0, 0);
79              const end = new Position(document.lineCount, 0);
80              const fullRange = new Range(start, end);
81              editBuilder.delete(fullRange);
82              editBuilder.insert(editor.selection.start, cilContent);
83          });
84
85      } catch (err) {
86          window.showErrorMessage('Failed to fetch and display CIL data: ' + err.message);
87          console.error('Error fetching CIL data:', err);
88      }
89  });
```

Command name

Method name

The commands call functions of the LSP API (sendRequest(), sendNotification(), etc.) to interact with the server. In the example in figure 9, we have created a method called "displayCIL", that is a request sent from the client to the server. The method logic is implemented in the server. We choose to name the command the same name as the method called inside.

[9]https://github.com/microsoft/vscode/blob/main/extensions/cpp/syntaxes/c.tmLanguage.json

3 Design Choices

3.1 File parsing

In order to use the Frama-C module's functions, the plugin converts the source code files into their C Intermediate Language (CIL) representation, which provides a detailed hierarchical structure of the source code. It captures the syntactic elements and relationships within the code, for deeper analysis and processing. This representation allows the server to efficiently navigate and manipulate the code, supporting features like 'Go To Definition' and other code navigation tools.

The system is designed to handle the parsing of both single and multiple source code files. Regardless of the number of files being parsed, the resulting ASTs from each file are merged into a single one. This merging process is crucial as it simplifies the analysis and manipulation of the code, providing a unified structure that the server can work with.

When dealing with large codebases, users often face the challenge of working with a vast number of source files. Parsing all these files can be a time-consuming task. To address this issue, the system offers a client setting ('sourceFiles') that allows users to specify which source files need to be parsed. By focusing the parsing process on only the relevant files, this setting helps optimizing the time spent on parsing, making the workflow more efficient.

Users must ensure that all necessary source files are included in the parsing process. If a user fails to specify a source file that is needed for certain features, such as Go To Definition, they may not function correctly. It is therefore the responsibility of the user to provide all the relevant files.

3.2 Language Server architecture

The server was implemented as a Frama-C plugin to fully exploit its advanced parsing and analysis capabilities.

The plugin operates in two distinct modes: command line mode and handler mode. In command line mode as represented in figure 10, the plugin functions independently of an editor, allowing for non-LSP usage. In this mode, the results are returned directly in the terminal as a JSON string. However, users must be cautious when combining options, as certain combinations may lead to errors or unintended results. Handler mode as depicted in figure 11, on the other hand, enables LSP functionality. In this mode, the plugin connects to a server process that listens on a specified port and initiates an infinite loop where a handler manages incoming and outgoing requests. The commands executed are determined by the handler in accordance with the user's workspace configurations.

The plugin consists of four components:

- **Cmdline_parameters**: Registers plugin, defines the plugin's command line options and the actions associated with them according to their values.
- **Handler**: Contains the code that connects to the editor, the main loop that handles requests, and stores the user's workspace configurations (sent by the editor).
- **Lib**: OCaml modules that implement the LSP types.
- **Features**: Contains the main features, functions interacting with the AST and other plugins to provide results.

3.3 Communication

Unix sockets were selected as the communication mechanism due to their efficiency and simplicity in local inter-process communication (IPC). Unix sockets provide a versatile and reliable means of exchanging data between processes running on the same machine.

Since the processes are all located on the same host, Unix sockets offer a significant performance advantage by avoiding the costs associated with network protocol layers, which are unnecessary in this context.

Furthermore, the OCaml Unix module, which provides native support for Unix sockets, allows ideal integration with the existing OCaml-based infrastructure of the server. This compatibility ensures that data is transmitted efficiently. By leveraging Unix sockets, we achieve a communication mechanism that is not only fast and reliable but also well-suited to the architecture of the application.

3.4 VS Code subprocess

The language server in this context is not an actual server in the traditional sense, but rather a process that connects to the VS Code server. VS Code manages the communication channels, such as pipes or sockets, by

Figure 10: LSP plugin example of command line usage sequence diagram

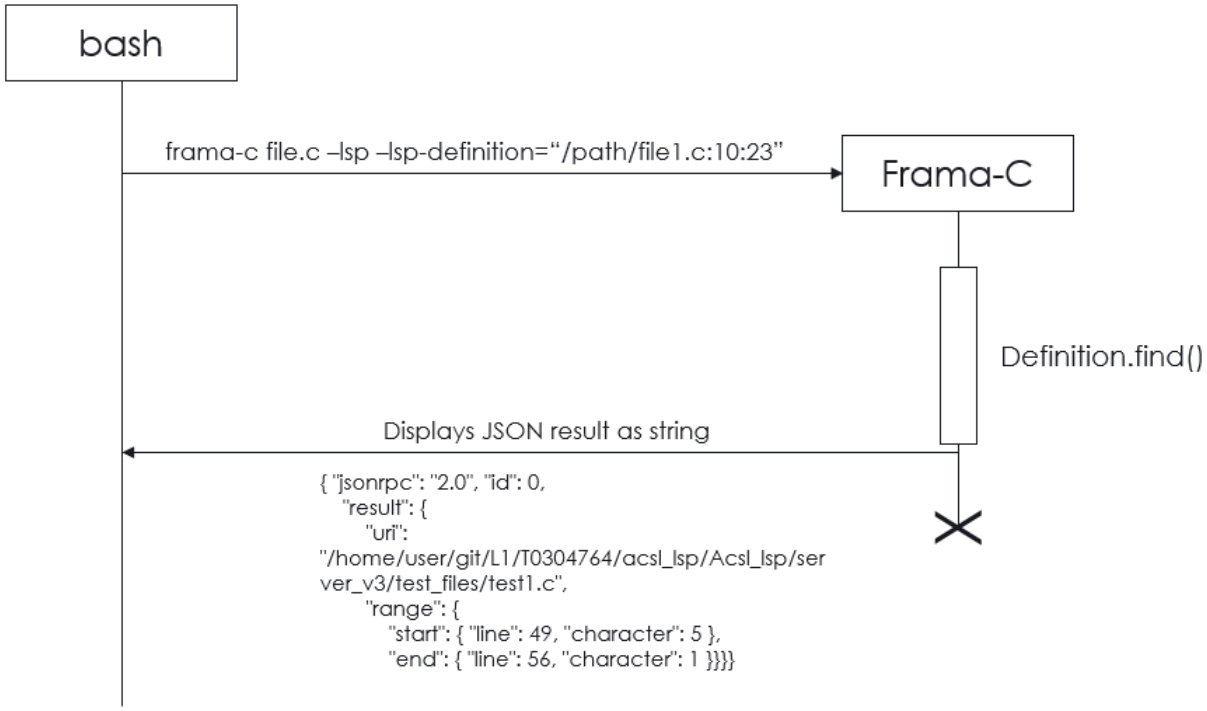
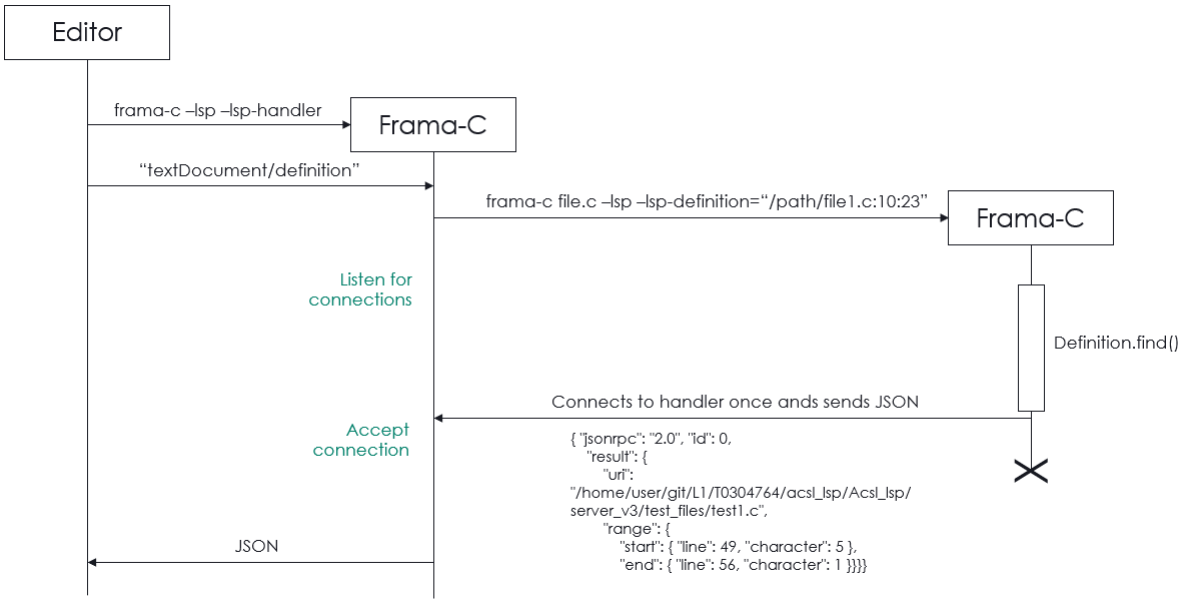


Figure 11: LSP plugin example of handler mode usage sequence diagram



creating them and then allowing the language server process to connect. Essentially, VS Code handles the setup by calling a function starting the server process, and ensuring the connection is established.

Once connected, the two processes can exchange messages. This setup ensures smooth communication, even if the language server is implemented in a different programming language. The Language server is launched via a shell script provided to the client, it runs the following command: 'frama-c -lsp -lsp-debug=4 -lsp-handler', the main Frama-C process.

This setup is specific to how VS Code manages extensions. Other editors might handle this connection differently, with varying approaches to managing server processes and communication channels. Only one

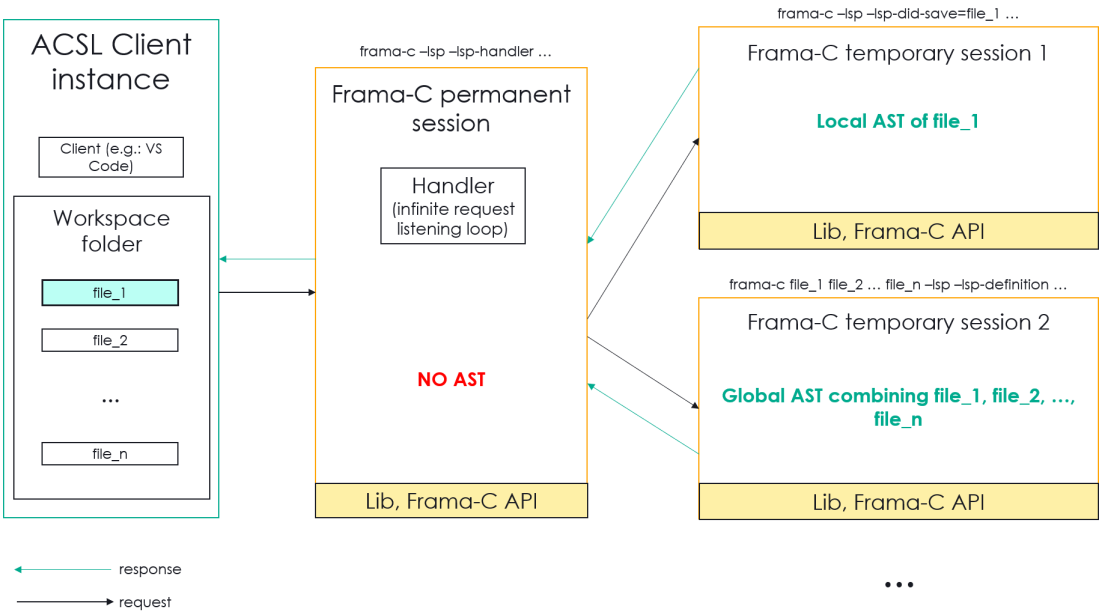


Figure 12: Architecture of the plugin

socket can connect to the VS Code subprocess, meaning that, if another entity were to connect to that port while it is running, its connection will be refused.

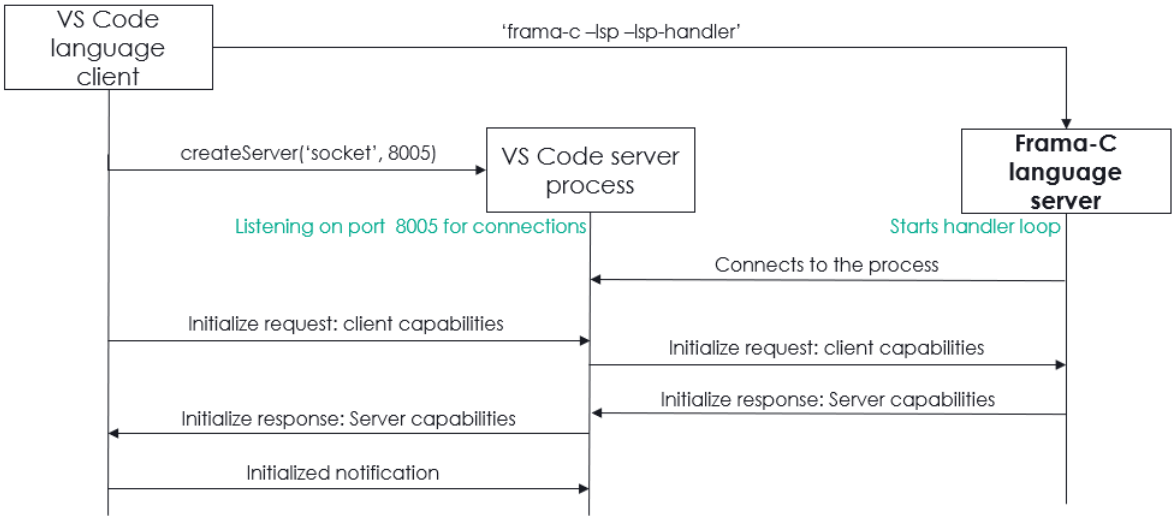


Figure 13: VS Code extension execution and first request exchanges between language client and server

3.5 Handler

The handler is executed in an infinite loop, continuously waiting for incoming requests. It uses the Unix recv function, which blocks the process until a request is received. This ensures that the handler remains inactive and consumes minimal resources until an LSP request or notification arrives, at which point it processes the request, sends the response, and then returns to waiting for the next input.

The handler first initializes the server capabilities, which define what the server can do (refer to 2.1.3).

3.5.1 Command Execution

For each LSP request (e.g., `textDocument/definition`, `textDocument/declaration`), the handler constructs a command line to invoke Frama-C with the appropriate options and parameters. These commands are built based on the request's content, such as file paths, line numbers, and character positions and also the current workspace, files being edited, and various global and local parameters (like include paths, macros, other frama-c

plugin options, etc.). The handler then executes the command and processes the output, which it sends back to the client as a response.

If an error occurs during the execution of a command, the handler catches the exception and returns an appropriate LSP error response.

When the result of a file parsing must be used by another plugin (e.g.: Callgraph, Metrics) Figure 3 below is a table listing an example of Frama-C command for each feature.

Feature	**Command**
Definition	`frama-c test\_files/test1.c -cpp-extra-args="-Itest_files include_folder -Itest_files/include_folder2" -lsp -lsp-debug=0 -lsp-definition="/home/user/git/L1/T0304764/ acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c:88:8"`
Declaration	`frama-c test_files/test1.c -cpp-extra-args="-Itest_files/ include_folder -Itest_files/include_folder2" -lsp -lsp-debug=0 -lsp-declaration="/home/user/git/L1/T0304764/ acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c:89:12"`
Display CIL	`frama-c test_files/folder1/test2.c -cpp-extra-args="- Itest_files/include_folder -Itest_files/include_folder2 " -lsp -lsp-debug=0 -lsp-display-cil`
Callgraph	`frama-c test_files/test1.c -cpp-extra-args="-Itest_files/ include_folder -Itest_files/include_folder2" -then -cg ="/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/ server_v3/test_files/test1.dot" -cg-services -then -lsp -lsp-debug=0 -lsp-compute-cg="/home/user/git/L1/ T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1"`
Local Metrics	`frama-c test_files/test1.c -cpp-extra-args="-Itest_files/ include_folder -Itest_files/include_folder2" -then - metrics -mtrics-by-function -metrics-output="/home/user /git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files /test1.txt" -then -lsp -lsp-debug=0 -lsp-metrics="/home /user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/ test_files/test1"`
Global Metrics	`frama-c test_files/test1.c test_files/folder1/test2.c -cpp- extra-args="-Itest_files/include_folder -Itest_files/ include_folder2" -then -metrics -mtrics-by-function - metrics-output="/home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/project_metrics.txt" - then -lsp -lsp-debug=0 -lsp-metrics="/home/user/git/L1/ T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/ project_metrics"`

Proof Obligations	`frama-c /home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c -cpp-extra-args="-I/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/include_folder -I/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/include_folder2" -then -lsp -lsp-root-path="/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files" -lsp-show-povc ="/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c:29:27" -wp -wp-gen`
Prove Annotation	Not implemented
Diagnostics	`frama-c -cpp-extra-args="-I/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server/test_files/include_folder -I/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server/test_files/include_folder2" -lsp -lsp-debug=4 -lsp-did-save=/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server/test_files/test1.c`

Table 2: Example Frama-C command for each feature

3.5.2 Example Workflow for `textDocument/definition`

3.5.3 Custom Features

The handler includes custom commands such as `displayCIL`, `showPOVC`, and metrics-related commands, which are specific to Frama-C.

Figure 3 below is a table listing an example of JSON result for each feature.

Feature	JSON
Definition	`{ "jsonrpc": "2.0", "id": 0, "result": { "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c", "range": { "start": { "line": 49, "character": 5 }, "end": { "line": 56, "character": 1 } } } }`
Declaration	`{ "jsonrpc": "2.0", "id": 0, "result": [ { "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/include_folder/test3.h", "range": { "start": { "line": 3, "character": 0 }, "end": { "line": 3, "character": 3 } } } ] }`
Display CIL	`{ "jsonrpc": "2.0", "id": 0, "result": "/* Generated by Frama-C */\n/*@ requires z > 7; \n ensures \\old(z) > 48; */\nfloat test2(float z)\n{\n float __retres;\n __retres = z + (float)48;\n return __retres;\n}\n\n\n" }`

Callgraph	{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Computed callgraph successfully, files generated : /home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.dot and /home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.c" } },
Local Metrics	{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/test1.txt" } },
Global Metrics	{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/test_files/project_metrics.txt" } }
Proof Obligations	{ "jsonrpc": "2.0", "id": 6, "result": "Goal Loop assigns (file test1.c, line 30) (3/3):\nEffect at line 35\nLet x = to_uint64(1 + i).\nLet a_1 = shift_sint32(a, 0).\nLet a_2 = havoc(Mint_undef_0, Mint_0, a_1, n).\nLet a_3 = shift_sint32(a, i).\nAssume {\n Type: is_uint64(i) /\\ is_uint64(n).\n (* Heap *)\n Type: (region (a.base) <= 0) /\\ linked(Malloc_0).\n (* Goal *)\n When: !invalid(Malloc_0, a_3, 1).\n (* Pre-condition *)\n Have: valid_rw(Malloc_0, a_1, n).\n (* Invariant *)\n Have: 0 <= n.\n (* Invariant *)\n Have: (0 <= i) /\\ (i <= n).\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < i) ->\n (a_2[shift_sint32(a, i_1)] = 0))).\n (* Then *)\n Have: i < n.\n (* Invariant *)\n Have: x <= n.\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < x) ->\n (a_2[a_3 <- 0][shift_sint32(a, i_1)] = 0))).\n}\nProve: included(a_3, 1, a_1, n).\n\n----------\nGoal Loop assigns (file test1.c, line 30) (2/3):\nEffect at line 34\nProve: true.\n\n----------\nGoal Loop assigns (file test1.c, line 30) (1/3):\nProve: true.\n" }
Prove Annotation	Not implemented
Diagnostics	{ "jsonrpc": "2.0", "method": "textDocument/publishDiagnostics", "params": { "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server/test_files/test1.c", "version": null, "diagnostics": [] } }

Table 3: Example of corresponding JSON result for each feature

Option	Type	Description	Default
-lsp	boolean	Enables LSP plugin	Off

-lsp-handler	boolean	Enables handler mode for LSP, (only to be used when launching the language server for LSP)	Off
-lsp-cmdline	boolean	Enables command line mode for LSP IF handler mode is activated, not to be used outside the LSP context	On
-lsp-id	integer	Id of the request, required if LSP handler mode is activated	0
-lsp-root-path	string	Path to the workspace folder opened in the editor, required if LSP handler mode is activated	“”
-lsp-compute-cg	string	Callgraph output file, to inform the user the files that were generated, write file name without extension	“”
-lsp-declaration	string	File, line and character where the user requested the declaration	“”
-lsp-definition	string	File, line and character where the user requested the definition	“”
-lsp-did-close	string	Name of the file to be closed, generates diagnostics. Do not give the filename to frama-c first.	“”
-lsp-did-save	string	Name of the file to be saved, generates diagnostics. Do not give the filename to frama-c first.	“”
-lsp-display-cil	boolean	If activated, sends the prettified CIL Ast of the parsed file	Off
-lsp-metrics	string	If specified, writes metrics inside the file. If only one file was parsed, outputs the metrics of that file, if multiple files were given, outputs the combined metrics of those files.	“”
-lsp-show-povc	string	File, line and character where the user requested the proof obligation(s). If the user wants the obligations of a pre-condition, they must send the location of the desired function call	“”

Table 4: List of Frama-C LSP plugin options

3.6 A first approach on the server implementation

In the context of Frama-C, an execution instance, whether initiated via the command line or through a function call within the Frama-C library, is referred to as a session. Initially, we considered implementing the server directly as a Frama-C plugin, where the main loop responsible for handling LSP requests would be executed within this plugin. This approach would handle all LSP requests into a single, continuous session.

However, this design presented a significant drawback: when Frama-C encounters a syntax error during file parsing, the entire session enters an error state. As mentioned in 2.2, if the AST is parsed again in the same session, it becomes corrupted. Given that a session in this context would persist until the server either crashes or the client terminates it, the error state would persist indefinitely. This is problematic because even if the user corrects the error in the code, the session—and consequently the server—would remain in the error state.

Terminating the session as a response to an error is not a viable solution, as it would lead to the server's shutdown. This, in turn, would cause the client to disconnect, leading to a crash of the extension. Consequently, the user would need to manually restart the extension, which is an undesirable outcome. To avoid such disruptions and maintain a seamless user experience, it is imperative that the server architecture be designed to handle errors more gracefully without necessitating a session termination.

4 Development

4.1 Project configurations

The user has the possibility to specify configuration values, for instance the names of the source files being processed, include paths and macros.

Table 5 lists all client configurations that are supported by the language server. They must have the same name for every client implementation.

Client configuration name	**Plug-in**	**Frama-C option(s)**
acslLsp	ACSL LSP	-lsp-debug

kernel.includePaths	Kernel	-cpp-extra-args
kernel.macros	Kernel	-cpp-extra-args
kernel.sourceFiles	Kernel	
kernel.machdep	Kernel	-machdep
kernel.keepUnusedSpecifiedFunctions	Kernel	-remove-unused-specified-functions
kernel.aggressiveMerging	Kernel	-aggressive-merging
kernel.generatedSpecCustom	Kernel	-generated-spec-custom
metrics.output	Metrics	-metrics, -metrics-output
callgraph.output	Callgraph	-cg, -cg-roots
callgraph.roots	Callgraph	-cg, -cg-roots
callgraph.services	Callgraph	-cg, -cg-no-services
wp.rte	WP	-wp, -wp-rte
wp.checkMemoryModel	WP	-wp, -wp-check-memory-model
wp.volatile	WP	-wp, -wp-volatile
wp.noPruning	WP	-wp, -wp-no-pruning
wp.prover	WP	-wp, -wp-prover
wp.timeout	WP	-wp, -wp-timeout
wp.session	WP	-wp, -wp-session
diagnostics.wp	ACSL LSP	-lsp-wp

Table 5: Frama-C options available to the user

4.2 Standard LSP features

4.2.1 Go To Definition

The 'Go To Definition' feature allows developers to easily navigate to the definition of symbols within their code.

When a user requests to find the definition of a symbol at a specific position in the code, the language server first identifies the name of the symbol located at that position. This involves parsing, analyzing the file and extracting the relevant word based on the provided line and column numbers. The code then initiates a search through various subsections of the code to find where the symbol is defined. This search involves visiting different parts of the source files' AST and concerns both ACSL and C code:

- ACSL Definitions: The code checks if the symbol matches any logical predicates or terms, which are specific to tools like Frama-C used for static analysis of C programs.
- C Definitions: It examines global entities, such as enums, structures, functions, and variables, to see if the symbol matches any of these.

Each of these searches uses visitor patterns, which traverse and inspect nodes in the AST, comparing each node's name to the symbol's name.

If a definition is located, the exact file path and the range of lines and columns where the symbol is defined are written into a JSON response. If no definition is found, a response indicating a null result is returned. This response allows the LSP client (the user's text editor or IDE) to navigate directly to the symbol's definition or handle the case where no definition exists.

4.2.2 Go To Declaration

The 'Go To Declaration' feature allows developers to easily navigate to the declaration of symbols within their code.

The process of finding declarations is the same algorithm as finding definitions except a few differences:

- The first visitor which visits all the global entities, traverses the AST and examines variable, function, enumeration and type declarations instead of definitions.
- An additional visitor is used to find variable declarations within a file. It visits the AST for l-values[10].
- If multiple declarations exist in header files, it will only send back the declaration that is located in the first header file included.
- If a variable is declared twice (one globally and one inside a function), no declaration is returned
- The plugin has the ability to send multiple declarations but it was omitted as Frama-C doesn't provide multiple declaration locations.

[10]L-values are nodes in the AST that are not functions, expressions or constants

4.2.3 File Synchronization & Diagnostics

File synchronization is a key feature of the LSP. Requests can by sent by the editing tool to notify document opening, changing and saving actions performed by the user. A single file parsing is performed each time a file is opened or saved. We add a Frama-C Kernel event listener to catch any error or warning raised by the Frama-C parser and we store them inside a JSON block. All diagnostics are published to the editor and displayed to the user in the form of squiggles and listed in the 'Problems' view of the editor.

There are two types of diagnostics: warnings and errors. Warnings are raised in case an ACSL annotation is not formed properly or when an unknown function is called. Errors are raised when there is a syntax error in the C code preventing the parser to finish (which are fatal errors). In the latter case, normally, any Frama-C session stops, but we ensure that diagnostics are sent either in case the parsing succeeds or fails. So we send them just before the exception aborts fatally. In case the parsing succeeds without any messages, the diagnostics are cleared in the editor.

The listener is a Frama-C function that reports any event emitted by Frama-C while parsing. From those events we can extract their source, their message and their kind (whether it is an error, a warning a feedback, etc.).

This feature is only implemented for single file parsing and diagnostics are computed only when the file is saved. No diagnostics are published for multiple file parsing, which means we do not know if those files are in state of error. So the user has to make sure the source files they provided have no errors.

4.3 Non-standard LSP features

Here is a description of features specific to Frama-C that are not specified by the LSP. To enable these features, they must be registered in the client, in the same way we did with VS Code.

4.3.1 Display CIL

This feature is designed to fetch and display the CIL representation of a C code file within the VS Code editor. This feature provides developers with a lower-level, intermediate view of the code, which is useful for understanding how the code is transformed and processed by Frama-C. It is particularly useful for developers who need to get an overview the intermediate representation of their C code, which is helpful for debugging. For example, when working on software verification projects, understanding the CIL can help in identifying how high-level constructs are lowered and managed internally by Frama-C. The possibility to see this representation directly in VS Code, with syntax highlighting, makes it easier to navigate and analyze it.

On the client side, the displayCIL command is registered to allow users to request the CIL representation of the currently active document by right-clicking on the editor and selecting the 'Display CIL' menu. When triggered, this command sends a request to the Frama-C language server, passing the active document's filename.

The server-side logic for displayCIL begins by constructing and executing a Frama-C command that includes options for generating the CIL and identifying the request by its ID. If successful, the server retrieves the AST of the code, converts it into a pretty-printed string representing the CIL, and sends this result back to the client.

Upon receiving the CIL data, the VS Code client creates a new untitled document in the editor, displays the CIL content in a new tab, and sets the document's language to ACSL for appropriate syntax highlighting. This allows the developer to view and analyze the CIL representation directly within the editor. If any errors occur during the process, they are reported back to the user through an error message, and the details are logged for debugging.

Figure 14 shows CIL representation in the right tab of the source code in the left tab.

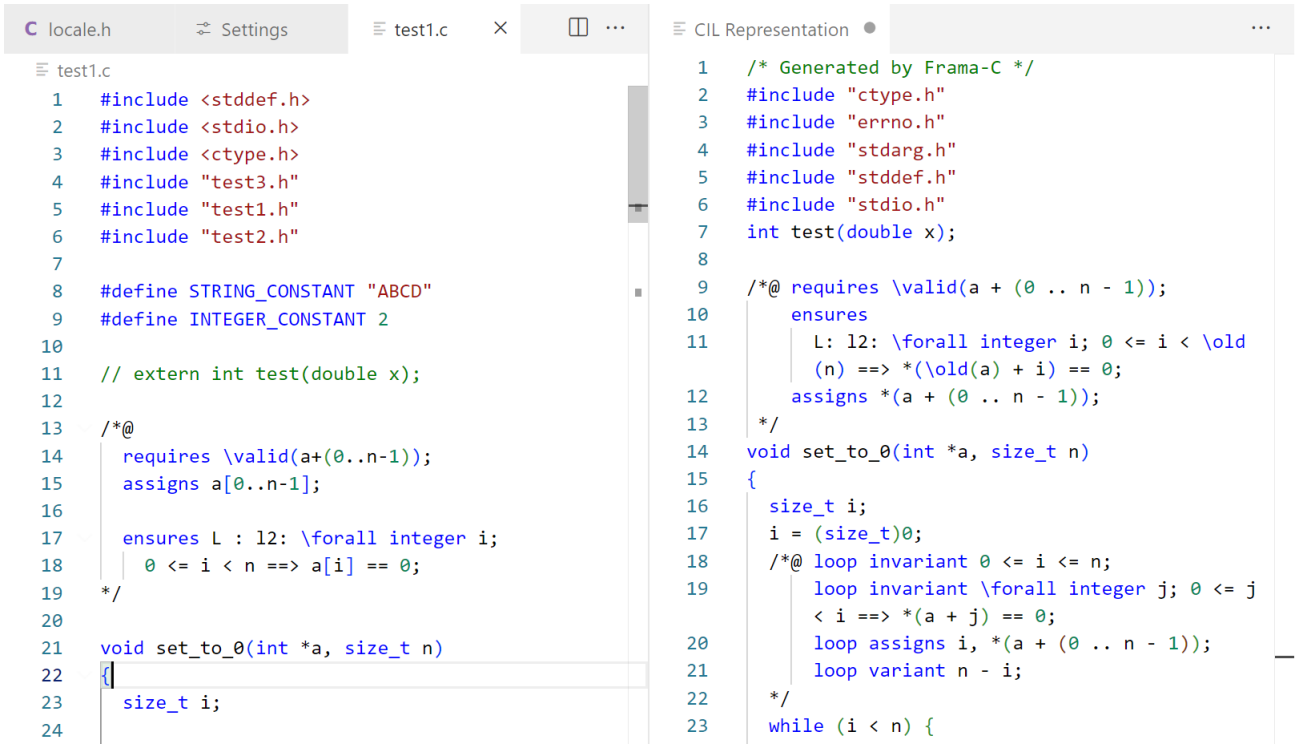
4.3.2 Callgraph

This feature is designed to compute and visualize the function call relationships within a given C source code. This feature generates a callgraph that maps out which functions call which other functions, providing a graphical representation of the code's structure. The callgraph can be helpful for understanding the control flow and dependencies within the program, especially in complex codebases. It makes it easier to identify key functions, understand dependencies, and determine the impact of potential changes. For example, if a developer wants to modify a core function, they can use the callgraph to see all the other functions that depend on it, allowing them to plan the changes more effectively. The generated .pdf file provides a clear representation of the callgraph.

The 'computeCG' method, defined in the client handles the generation of the callgraph. It starts by extracting the relevant file from the parameters provided by the LSP notification. The plugin then constructs a command to invoke Frama-C, passing the necessary file paths and configuration options, including arguments specific to callgraph computation (callgraph_args). This command directs Frama-C to analyze the code and produce a .dot file that describes the callgraph in Graphviz format.

On the client side, the 'computeCG' command is registered to trigger the callgraph computation. When invoked, this command sends a notification to the Frama-C language server, passing the filename of the currently

Figure 14: Example of execution of 'Display CIL' in VS Code



active document in the editor. The notification is sent using client.sendNotification, which initiates the process on the server.

4.3.3 Metrics

The metrics feature of the Frama-C plugin enables the calculation and retrieval of both global and local metrics for a C project, providing developers with insights into the code's complexity, structure, and other relevant details. It is integrated into the Frama-C language server and is accessible through the VS Code client.

On the VS Code client side, two commands are registered: showLocalMetrics and showGlobalMetrics. These commands allow users to request either local metrics for the currently active file or global metrics for the entire project (files specified by the user or all source files in the workspace folder). When these commands are invoked, they send notifications to the Frama-C language server.

The server processes these requests by constructing and executing Frama-C commands with appropriate arguments. For global metrics, the command analyzes the entire project and stores the results in a specified file, while for local metrics, the command focuses on the active file. The results are saved as text files, typically named after a hard-coded name (global) or the name of the file being analyzed (local).

Once the metrics are calculated, the server notifies the client, indicating that the files have been generated. If any issues occur during the process, they are communicated back to the user through error messages.

4.3.4 Proof Obligations

This feature provides users the possibility to inspect specific proof obligations related to functions in their code, within the context of ACSL annotations. If a developer is working on a critical section of code and needs to verify that all preconditions and assertions are properly addressed. Using the proof obligations feature, the developer can quickly spot the exact proof obligations generated by ACSL annotations for a particular function. By reviewing these obligations in VS Code, the developer can ensure that the necessary proofs are in place.

On the VS Code client side, the showPOVC command is registered, enabling users to request proof obligations for the currently selected function or code location in the editor. When the command is invoked, it sends the file name and cursor position to the Frama-C language server.

The server processes this request by extracting relevant proof obligations from the specified location in the code. The Frama-C command is constructed with additional source files if the target file is a header file. The server then filters proof obligations based on their position in the file and returns the results to the client. If proof obligations are found, they are formatted as a JSON response.

On the client side, the response is parsed and displayed in a new editor tab within VS Code. The proof obligations are presented in a plaintext format, allowing the developer to review the conditions and assertions

that must be proven for the correctness of the code.

5 LSP use cases with VS Code

In this enclosed figure is a listing of all LSP features, if they are handled by VS Code, and their implementation status on the server side.

Feature	**Related Frama-C API modules**	**Main algorithm**	**Parsing mode**
Go To Declaration	Cil, Visitor, Filepath, File, Ast, Json	Tree traversal of AST	Global
Go To Definition	Cil, Visitor, Filepath, File, Ast, Json	Tree traversal of AST	Global
Diagnostics	Filepath, File, Log, Json	Log event listener	Local
Display CIL of file	Ast, Printer, Pretty_utils, Json	Get computed AST	Local
Show proof obligations (POs) of annotation(s)/file	WP, Property, Filepath, Json, Pretty_utils	List traversal of generated proof obligations	Local
Show proof verdict		List traversal	Local
Compute Callgraph	Callgraph	Get computed Callgraph	Local
Show Metrics	Metrics	Get metrics result	Local

Table 6: Features implementation details

LSP features	**Use case(s)**	**Implementation status**	**Supported by VS Code**
Language features			
Go to Declaration	Show declaration of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented	Yes
Go to Definition	Show definition of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented	Yes
Go to Type Definition	Show definition of C library, Frama-C and user defined functions types	Implemented (included in go to definition)	Yes
Go to Implementation	Show implementation of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.		Yes
Find References	Show references of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.		Yes
Prepare Call Hierarchy	Prepare call hierarchy of C functions, ACSL terms and predicates		Yes
Call Hierarchy Incoming Calls	List incoming calls of C functions		Yes
Call Hierarchy Outgoing Calls	List outgoing calls of C functions		Yes
Prepare Type Hierarchy	Prepare type hierarchy of C types		Yes
Type Hierarchy Super Types	List super types of C types		Yes

Type Hierarchy Sub Types	List sub types of C types		Yes
Document Highlight	Highlight occurrences of all words OR only C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates		Yes
Document Link	Shows links written inside C and ACSL comments and make them browsable		Yes
Document Link Resolve	Related to Document Link		Yes
Hover	Show C function contract, show C function type information and documentation if available		Yes
Code Lens	Show C function signature		Yes
Code Lens Refresh	Refresh C function signature if changes were made		Yes
Folding Range	Fold function contracts, functions, function descriptions, comments, ghost code		Yes
Selection Range	Suggest selection range of function contracts, functions, function descriptions, comments, ghost code		Yes
Document Symbols	List (in the current file) symbols of C functions, variables, ACSL terms and predicates.		Yes
Semantic Tokens	Request semantic tokens for C and ACSL keywords to provide additional color information.		Yes
Inline Value	Show, at the end of the line, the values of C variables in a line		Yes
Inline Value Refresh	Refresh the values of C variables in a line if changed		Yes
Inlay Hint	No use cases found		Yes
Inlay Hint Resolve	No use cases found		Yes
Inlay Hint Refresh	No use cases found		Yes
Moniker	No uses cases found		Yes
Completion Proposals	Show completion item for ACSL : ACSL keywords completion (ensures, assigns, etc.), propose variables bound to the current file or workspace,C keyword completion.	Ongoing but disabled	Yes
Completion Item Resolve	Extend Completion Proposals feature		Yes
Publish Diagnostics	Show compilation errors, warnings for C and ACSL code.		Yes
Pull Diagnostics	Related to Publish Diagnostics		Yes
Signature Help	Show C library, Frama-C and user defined function signatures when function name is typed		Yes
Code Action	The server can provide code actions related to a diagnostic such as fixing includes, etc.		Yes
Code Action Resolve	Related to Code Action		Yes
Document Color	Show color of ANSI color codes in the C code if any (when printing colors in the terminal, etc.)		Yes
Color Presentation	Related to Document Color		Yes

Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators.		Yes
Range Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators for selected lines in the code.		Yes
On type Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators as user types according to formatting trigger characters provided by the server. Trigger characters being ";" and "}" so that formatting is done after writing a line or a block.		Yes
Rename	Workspace-wide rename of a symbol (C functions, variables, macros, types, ACSL types, ACSL terms and predicates)		Yes
Prepare Rename	Related to Rename		Yes
Linked Editing Range	No use cases found yet		Yes
Workspace features			
Workspace Symbols	Helps users find and list symbols across a workspace based on search queries		Yes
Workspace Symbol Resolve	Related to Workspace Symbols		Yes
Get Configuration	Get workspace settings: include paths, source files, macros	Implemented	Yes
Did Change Configuration	Get configurations each time user changes them	Implemented	Yes
Workspace Folders	Allow support of multiple roots folders		Yes
Did Change Workspace Folders	Be notified each time a workspace folder structure is changed		Yes
Will Create Files	Get workspace edits which will be applied to workspace before the files are created		Yes
Did Create Files	Get notified when files are created from within the client		Yes
Will Rename Files	Get notified of file renames before files are actually renamed as long as the rename is triggered from within the client either by a user action or by applying a workspace edit		Yes
Did Rename Files	Get notified when files are renamed from within the client		Yes
Will Delete Files	Get notified of file deletes before deleting them		Yes
Did Delete Files	Get notified when file are deleted		Yes
Did Change Watched Files	Get notified when files outside the primary editing scope, such as makefiles or python/bash scripts related to the project		Yes

Execute Command	Generate boilerplate code, a function contract skeleton		Yes
Apply Edit	Automatically insert or update ACSL annotations in the code after a refactoring operation, ensuring that the formal specifications remain coherent with the modified code		Yes
Window features			
Show Message Notification	Notify the user when a file is generated by one of the plugins (callgraph, metrics, ...)	Implemented for callgraph and metrics	Yes
Show Message Request	If the user asked for the proof obligation of a pre-condition of a function, asks for which function call it should show the proof obligations		Yes
Log Message	No use case found		Yes
Show Document	Show generated callgraph		Yes
Create Work Done Progress	Display proof verdict calculation status bar		Yes
Cancel a Work Done Progress	Related to work done progress		Yes
Sent Telemetry	Enable the collection of usage and performance data to monitor server performance, analyze feature usage, track errors, and help development improvements		Yes

Table 7: LSP features use cases

VS Code features	**Use cases**	**Implementation status**
Programmatic		
Provide Diagnostics	Show compilation errors, warnings for C and ACSL code.	Implemented
Show Code Completion Proposals	ACSL keywords completion (ensures, assigns, etc.), propose variables bound to the current file or workspace, C keyword completion.	VS Code supports word based completions for any programming language
Show Hovers	Show C function contract, show C function type information and documentation if available	
Help With Function and Method Signatures	Show C library, Frama-C and user defined function signatures when function name is typed	
Show Definitions of a Symbol	Show definition of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented
Find All References to a Symbol	Show references of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	
Highlight All Occurrences of a Symbol in a Document	Highlight occurrences of all words OR only C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates	

Show all Symbol Definitions Within a Document	Show all symbols in the document. Define the kinds of symbols such as variables, functions	
Show all Symbol Definitions in Folder	Show all symbols define by the source code within the open folder. Define the kinds of symbols such as variables, functions	
Possible Actions on Errors or Warnings	Fix missing semi-colons for C and ACSL code, unclosed brackets and parenthesis	
CodeLens - Show Actionable Context Information Within Source Code	Show C function signature	
Show Color Decorators	Show color of ANSI color codes in the C code if any (when printing colors in the terminal, etc.)	
Format Source Code in an Editor	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators.	
Format the Selected Lines in an Editor	Same as above but for selected lines in the code.	
Incrementally Format Code as the User Types	Same as above but code formats automatically as user types according to formatting trigger characters provided by the server. Trigger characters being ";"and "}" so that formatting is done after writing a line or a block.	
Rename Symbols	Rename all occurrences of a symbol, it should differentiate between functions, predicates, terms, variables and constants as they can have the same name.	
Declarative		
Syntax highlighting		Implemented
Snippet completion		
Bracket matching		Implemented
Bracket autoclosing		Implemented
Bracket autosurrounding		Implemented
Comment toggling		Implemented
Auto indentation		Implemented
Folding		Implemented

Table 8: VS Code Programmatic and Declarative Features

6 Illustration of the implementation

7 Conclusion

Results & Feedback

Our main

Future work

– Enhance display of proof obligations: ask the user which function call's pre-condition to prove whenever proof obligations are requested on a pre-condition of a function definition

- Enhance declaration feature: find macros and find multiple declarations if they exist, return declarations within the right scope instead of all variables with the same name.
- Enhance definition feature: must be able to find Frama-C's standard library function definitions
- Implement display of proof verdict of annotation
- Implement autocompletion feature logic: necessary structures are provided but not the logic behind completion items searching
- Enhance WP diagnostics: if the user activates the configuration to enable WP diagnostics, shows WP diagnostics on top of the regular (kernel) diagnostics
- Enhance metrics feature by displaying results in new output console
- Enhance callgraph feature by displaying browsable callgraph on the editor
- Implement a 'server' mode for other editors chose communication methods are different from VS Code's
- Implement hover feature for ACSL keywords
- Implement request processing for large files
- Implement request queue with ability to cancel requests

Internship review

I have learned a lot etc etc

References

[1] Yannik Sander. "Design and Implementation of the Language Server Protocol for the Nickel Language". MA thesis. KTH, School of Electrical Engineering and Computer Science (EECS), 2022. URL: https://kth.diva-portal.org/smash/get/diva2:1732389/FULLTEXT01.pdf.

[2] Loïc Correnson et al. *Frama-C User Manual*. URL: http://frama-c.com/download/frama-c-user-manual.pdf.

[3] Julien Signoles et al. *Frama-C Plug-in Development Guide*. URL: http://frama-c.com/download/frama-c-plugin-development-guide.pdf.